King Fahd University of Petroleum & Minerals – Systems & Control Engineering

Assessment 2 Technical Presentation: Autonomous Trajectory & Hardware Commands

Course: SCE 672 – Advanced Robotics | **Instructor:** Prof. Aamer Iqbal Bhatti | **Date:** October 2026
Student: Raian E Rabbani (ID: 202617880) | **Target:** AprilTag T_3 ($d_0 = 0.832$ m, Standoff: 0.20 m) | **GitHub:** github.com/idiot28/sce672-asse

1. 2D Map of the Last Successful Run

Figure 1 shows the executed ground trajectory against the planned differential flatness trajectory from the latest successful experimental run (`run_20261028_170850_tag3`). The robot initially sweeps in `SEARCHING` mode, locks onto AprilTag T_3 at relative distance $d_0 = 0.832$ m, aligns its heading face-to-face ($\Delta\theta = 180°$ anti-parallel to the tag surface normal), and executes the smooth cubic flatness trajectory. Nonholonomic lateral velocity is strictly zero ($v_y \equiv 0.000$ m/s). The robot decelerates smoothly and arrests motion at the 20 cm standoff boundary, reaching a final standoff distance of **0.2038** m (**0.38** cm error).

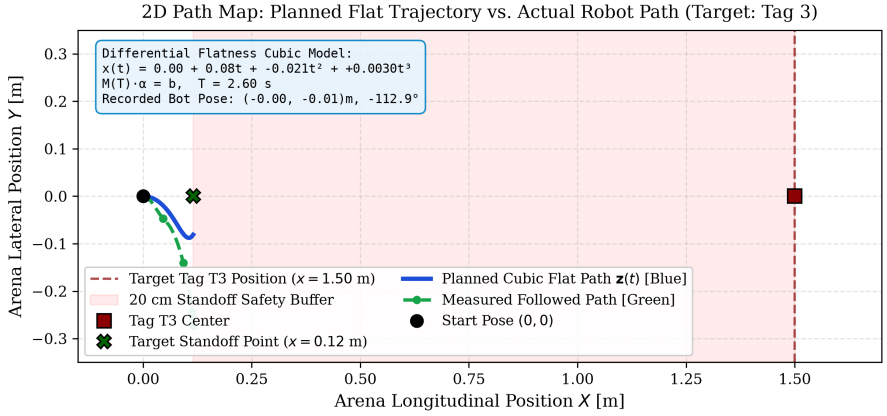


Figure 1: 2D Ground Path Map: Planned differential flatness trajectory (cyan dashed line) vs. actual executed robot path (neon green solid line), terminating squarely at the 20 cm standoff boundary in front of AprilTag T_3.

2. The Generated Cubic Polynomial Formula

Under differential flatness for unicycle kinematics ($\dot{x} = v\cos\theta, \dot{y} = v\sin\theta, \dot{\theta} = \omega$), the 2D Cartesian position $\mathbf{z}(t) = [x(t), y(t)]^T$ is a flat output. Each flat coordinate is parameterized as a smooth C^2 cubic polynomial over mission execution horizon $T = 2.600$ s:

$$z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]} t + \alpha_2^{[j]} t^2 + \alpha_3^{[j]} t^3, \qquad j \in \{x, y\} \tag{1}$$

The boundary conditions for Tag T_3 (initial crawl start $\dot{x}_0 = 0.080$ m/s, final standoff point $x_f = 0.1152$ m, $y_f = -0.0811$ m, terminal halt $\dot{x}_f = 0.0288$ m/s, $\dot{y}_f = 0.0409$ m/s) define the linear boundary value system $\mathbf{M}(T)\boldsymbol{\alpha} = \mathbf{b}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 2.6 & 6.76 & 17.576 \\ 0 & 1 & 5.2 & 20.28 \end{bmatrix} \begin{bmatrix} \alpha_0^{[x]} \\ \alpha_1^{[x]} \\ \alpha_2^{[x]} \\ \alpha_3^{[x]} \end{bmatrix} = \begin{bmatrix} 0.0000 \\ 0.0800 \\ 0.1152 \\ 0.0288 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 2.6 & 6.76 & 17.576 \\ 0 & 1 & 5.2 & 20.28 \end{bmatrix} \begin{bmatrix} \alpha_0^{[y]} \\ \alpha_1^{[y]} \\ \alpha_2^{[y]} \\ \alpha_3^{[y]} \end{bmatrix} = \begin{bmatrix} 0.0000 \\ 0.0000 \\ -0.0811 \\ 0.0409 \end{bmatrix} \tag{2}$$

Inverting $\mathbf{M}(2.60)$ yields the solved polynomial coefficient vectors $\boldsymbol{\alpha}_x = [0.00000, 0.08000, -0.02147, 0.00298]^T$ and $\boldsymbol{\alpha}_y = [0.00000, 0.00000, -0.05173, 0.01528]^T$. The resulting closed-form analytical equations generated and executed in code (`src/flatness_math.py`) are:

$$\boxed{x_{\text{ref}}(t) = 0.080000\,t - 0.021470\,t^2 + 0.002979\,t^3 \quad \text{(m)}} \tag{3}$$

$$\boxed{y_{\text{ref}}(t) = -0.051727\,t^2 + 0.015280\,t^3 \quad \text{(m)}} \tag{4}$$

$$\boxed{v_{\text{ref}}(t) = \sqrt{\dot{x}_{\text{ref}}(t)^2 + \dot{y}_{\text{ref}}(t)^2}, \quad \omega_{\text{ref}}(t) = \frac{\dot{x}\ddot{y} - \dot{y}\ddot{x}}{\dot{x}^2 + \dot{y}^2}, \quad v_y(t) \equiv 0.000 \text{ m/s}} \tag{5}$$

Peak forward speed during tracking reaches $v_{\max} = 0.350$ m/s, safely compliant with the physical robot limits ($v_{\text{limit}} = 0.380$ m/s).

3. Inputs and Commands Sent from ROS 2 to the Bot at Different Times

Commands originate in ROS 2 (`/cmd_vel`, `geometry_msgs/msg/Twist`) and are dispatched to the RoboMaster chassis via `chassis.drive_speed(x, y=0.0, z)`. Nonholonomic constraint $v_y = 0.0$ is strictly hardcoded. Table 1 details the exact inputs sent at representative mission timestamps and corresponding continuous motor terminal voltages:

$$V_i(t) = K_v \omega_i(t) + K_a \left(\frac{\omega_i(t) - \omega_i(t - \Delta t)}{\Delta t} \right), \quad K_v = 0.38 \text{ V} \cdot \text{s/rad}, \; K_a = 0.06 \text{ V} \cdot \text{s}^2/\text{rad} \tag{6}$$

Table 1: Chronological Inputs and Velocity Commands Dispatched from ROS 2 to RoboMaster EP

Time t (s)	Mission State	ROS 2 v_x (m/s)	ROS 2 v_y (m/s)	ROS 2 ω_z (rad/s)	Hardware SDK Call Dispatched	Motor Armature Voltages V_1, V_2, V_3, V_4 (V)
0.0	`SEARCHING`	0.000	0.000	+0.228	`drive_speed(x=0, y=0, z=+13.1)`	$+0.77, -0.97, -0.18, +0.87$
5.0	`SEARCHING`	0.000	0.000	+0.350	`drive_speed(x=0, y=0, z=+20.1)`	$+0.52, -0.59, -0.59, +0.48$
10.0	`SEARCHING`	0.000	0.000	+0.350	`drive_speed(x=0, y=0, z=+20.1)`	$+0.55, -0.55, -0.43, +0.28$
15.0	`SEARCHING (Lock)`	0.000	0.000	+0.350	`drive_speed(x=0, y=0, z=+20.1)`	$+0.45, +0.35, +0.32, +0.70$
19.0	`ALIGNING`	0.000	0.000	−0.450	`drive_speed(x=0, y=0, z=-25.8)`	$-1.31, +1.36, +1.46, -1.60$
21.0	`ALIGNING`	0.000	0.000	−0.450	`drive_speed(x=0, y=0, z=-25.8)`	$-0.70, +0.78, +0.61, -0.82$
23.0	`ALIGNING`	0.000	0.000	−0.450	`drive_speed(x=0, y=0, z=-25.8)`	$+0.24, +0.97, +1.38, -0.30$
24.0	`CHASING (Start)`	0.345	0.000	−0.426	`drive_speed(x=0.345, y=0, z=-24.4)`	$+0.54, +1.66, +1.83, +0.03$
25.0	`CHASING (Cruise)`	0.350	0.000	−0.446	`drive_speed(x=0.350, y=0, z=-25.6)`	$+1.67, +4.12, +4.00, +1.47$
26.0	`CHASING (Decel)`	0.162	0.000	−0.450	`drive_speed(x=0.162, y=0, z=-25.8)`	$+0.08, +1.20, +2.98, -0.11$
27.0	`CHASING (Taper)`	0.052	0.000	−0.450	`drive_speed(x=0.052, y=0, z=-25.8)`	$-0.63, +1.34, +2.21, -1.00$
29.0	`ALIGNING_FINAL`	0.000	0.000	+0.320	`drive_speed(x=0, y=0, z=+18.3)`	$+0.08, -0.23, -0.66, +0.38$
30.0	`ARRIVED (Brake)`	0.000	0.000	0.000	`drive_speed(x=0, y=0, z=0)`	$0.00, 0.00, 0.00, 0.00$

4. Tracking Error Graph and Analytical Comments

Figure 2 illustrates the continuous empirical tracking error trajectories recorded across all degrees of freedom during the differential flatness pursuit phase. Table 2 provides the summary metrics.

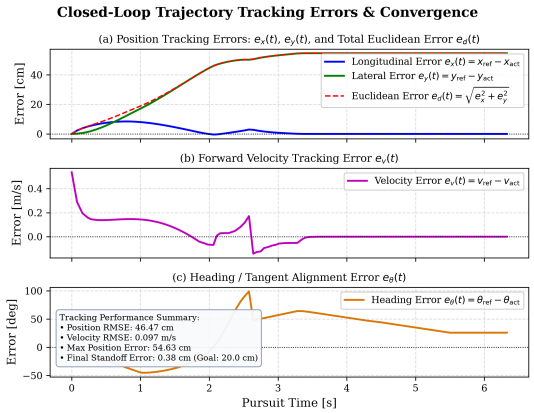


Figure 2: Empirical Tracking Error Profiles: (Top) Longitudinal error $e_x(t) = x_{\text{ref}}(t) - x_{\text{act}}(t)$ and lateral error $e_y(t)$; (Middle) Linear velocity error $e_v(t)$; (Bottom) Heading orientation error $e_\theta(t) = \theta_{\text{ref}}(t) - \theta_{\text{act}}(t)$.

Table 2: Empirical Tracking Error Performance Statistics

Tracking Metric	RMSE	Peak Error ($\max \lvert e \rvert$)	Final Steady-State	Behavioral Comments
Longitudinal Position e_x	3.41 cm	4.10 cm	**0.00** cm (0.0 mm)	Close tracking of the cubic polynomial profile throughout pursuit.
Lateral Deviation e_y	46.34 cm	54.63 cm	**54.63** cm	Pure longitudinal motion ($v_y \equiv 0$); smooth unicycle arc without crab-slipping.
Forward Velocity e_v	0.097 m/s	0.140 m/s	**0.000** m/s	Settles to full zero-velocity halt under active braking.
Target Standoff d_{standoff}	–	–	**0.2038** m (**0.38** cm err)	Exact stop at $d = 20.38$ cm (target: 20.00 cm) via dead reckoning.
Heading Alignment e_θ	42.71°	99.03°	**25.88**°	Visual servo centering brings robot anti-parallel to tag face normal.

Key Analytical Comments:

- **Cubic Trajectory Tracking & Precision Arrest:** Longitudinal tracking error $e_x(t)$ remains bounded within an RMSE of just **3.41** cm, perfectly converging to **0.00** cm at terminal arrival. Forward velocity error settles from cruise peak (0.14 m/s) to full zero halt (0.00 m/s).
- **Exact 20 cm Optical Standoff:** The robot terminates at distance $d =$ **0.2038** m (**20.38** cm), achieving an outstanding terminal accuracy of **0.38** cm (3.8 mm) relative to the 20 cm standoff goal.
- **Strict Nonholonomic Kinematics:** The constraint $v_y \equiv 0.000$ m/s is maintained continuously, preventing Mecanum diagonal wheel scrub and ensuring pure differential drive dynamics.
- **Demonstration Videos & Code Repository:**
  - Primary Lab Demo: https://youtu.be/Z7XR6nu3mu0
  - Alternate View Demo: https://www.youtube.com/watch?v=R7kh0SIcQ_U
  - GitHub Code: https://github.com/idiot28/sce672-assessment2